

ЛАБОРАТОРНАЯ РАБОТА 5

1. Назначение и краткая характеристика работы

В процессе выполнения настоящей работы предстоит ознакомиться с использованием языка Python для многопоточного программирования.

Целью работы является приобретение навыков в разработке многопоточного приложения.

1. Задание на работу в лаборатории

1. Переписать код лабораторной работы №3 таким образом, чтобы расчет схем, представленных в формате JSON, происходил в отдельных потоках.
2. Измерить время выполнения программы по расчету как минимум трех схем с применением многопоточного программирования и без него.

2. Методические указания к работе в лаборатории

Поток — это минимальная единица работы, запланированная для выполнения операционной системой.

Свойства потоков:

- они существуют внутри процесса;
- в одном процессе может быть несколько потоков;
- потоки в одном процессе разделяют состояние и память родительского процесса.

Работу с несколькими потоками реализуют такие библиотеки в Python, как `threading` и `multiprocessing`.

`Threading` — параллелизм на основе потоков. Этот модуль создает интерфейсы многопоточности более высокого уровня поверх `_thread` модуля более низкого уровня.

Класс `Thread` представляет действие, которое выполняется в отдельном потоке управления. Существует два способа указать действие: путем передачи вызываемого объекта конструктору или путем переопределения метода `run()` в подклассе. Никакие другие методы (кроме конструктора) не должны переопределяться в подклассе. Другими словами, только переопределяются методы этого класса `__init__()` и `run()`.

Как только объект `thread` создан, его действие должно быть запущено путем вызова `start()` метода `thread`. Это вызывает метод `run()` в

отдельном потоке управления. Как только активность потока запускается, поток считается ‘живым’. Он перестает быть активным, когда его `run()` метод завершается – либо обычным образом, либо путем создания необработанного исключения. Метод `is_alive()` проверяет, является ли поток активным.

Другие потоки могут вызывать метод `join()` потока. Это блокирует вызывающий поток до тех пор, пока поток, метод `join()` которого вызывается, не будет завершен.

У потока есть имя. Имя может быть передано конструктору и прочитано или изменено с помощью `name` атрибута.

Если метод `run()` вызывает исключение, `threading.excepthook()` вызывается для его обработки. По умолчанию `threading.excepthook()` игнорируется автоматически `SystemExit`.

Поток может быть помечен как “поток демона”. Значение этого флага заключается в том, что вся программа Python завершается, когда остаются только 3 потока демона. Начальное значение наследуется от создающего потока. Флаг может быть установлен с помощью `daemon` свойства или аргумента конструктора `daemon`.

`Multiprocessing` — параллелизм на основе процессов. `multiprocessing` это пакет, который поддерживает порождающие процессы с использованием API, аналогичного `threading` модулю. Пакет `multiprocessing` предлагает как локальный, так и удаленный параллелизм, эффективно обходя глобальную блокировку интерпретатора, используя подпроцессы вместо потоков. Благодаря этому `multiprocessing` модуль позволяет программисту полностью использовать несколько процессоров на данной машине. Он работает как в Unix, так и в Windows.

3. Задание к составлению исполнительного отчета

Исполнительный отчет должен включать в себя:

1. титульный лист;
2. листинг кода;
3. выводы о проделанной работе.